

D600 Task 1 - Aaron Pacheco

A. Describe the purpose of this data analysis by doing the following:

1. Propose **one** research question that is relevant to a real-world organizational situation captured in the provided dataset that you will answer using multiple linear regression in the initial model.

Research Question: How do housing features affect housing prices?

2. Define **one** goal of the data analysis. Ensure that your goal is relevant within the scope of the scenario and is represented in the available data.

The goal of this data analysis is to identify which housing features should be prioritized in future models for housing prices.

B. Summarize the data preparation process for multiple linear regression analysis by doing the following using Python or R:

1. Identify the dependent variable and **10** or more independent variables that are required to answer the research question and justify your selection of all variables.

Dependent Variable:

Price - Our goal is to determine what variables affect housing prices, so using Price as the dependent variable is the only logical choice.

Independent Variables:

Square Footage, NumBathrooms, NumBedrooms, BackyardSpace - These four variables are directly linked to the size of the home, which is commonly thought to have a major effect on housing prices. I expect that as these variables increase, the housing price will also increase.

CrimeRate, SchoolRating, DistanceToCityCenter, EmploymentRate - These four variables tell us about the area the home is in. It would be logical to believe that the area surrounding a home could have a direct effect on the price of a home. I expect a positive correlation between Price and both SchoolRating and Employment Rate. I expect a negative correlation between Price and both CrimeRate and DistanceToCityCenter

AgeOfHome, RenovationQualityRate - These two variables tell us about the quality of the build on the house. It seems logical to believe that the higher the RenovationQualityRate, the higher the Price. AgeOfHome may not provide as clear of a correlation, because older houses may be more run-down, but if they are well-kept, the antique value could alternatively fetch a higher price.

2. Provide at least **one** screenshot of the descriptive statistics (e.g., mean, median, range, standard deviation, variance, percentiles, quartiles) for the quantitative (i.e., numeric) variables

selected (e.g., frequency counts and percentages) from part B1 and for the qualitative (i.e., categorical) variables if selected.

```

D600 Task 1.py X D600 Task 2.py D600 Task 3.py
C:\Users\imret>Downloads>D600 Task 1.py>...
24 #Calculate descriptive statistics
25 stats = df[description_vars].describe()
26
27 #Add additional statistics not included in describe()
28 stats.loc['median'] = df[description_vars].median()
29 stats.loc['variance'] = df[description_vars].var()
30 stats.loc['range'] = df[description_vars].max() - df[description_vars].min()
31
32 #Rename labels
33 stats = stats.rename(index={
34     'count': 'Count',
35     'mean': 'Mean',
36     'std': 'Standard Deviation',
37     'min': 'Minimum',
38     '25%': '25th Percentile (Q1)',
39     '50%': '50th Percentile (Q2)',
40     '75%': '75th Percentile (Q3)',
41     'max': 'Maximum',
42     'median': 'Median',
43     'variance': 'Variance',
44     'range': 'Range'
45 })
46
47 print(stats)

```

	Price	SquareFootage	NumBathrooms	NumBedrooms	BackyardSpace	CrimeRate	SchoolRating	AgeOfHome	DistanceToCityCenter	EmploymentRate	RenovationQualityRate
Count	7.000000e+03	7000.000000	7000.000000	7000.000000	7000.000000	7000.000000	7000.000000	7000.000000	7000.000000	7000.000000	7000.000000
Mean	3.072820e+05	1048.947459	2.117857	3.008571	511.507029	31.226194	6.949571	46.797046	17.475337	93.711349	5.016857
Standard Deviation	1.501734e+05	426.010482	1.006889	1.021940	279.926549	18.025327	1.909931	31.779701	12.024985	4.505359	1.974766
Minimum	8.500000e+04	550.000000	1.000000	1.000000	0.390000	0.030000	1.000000	0.010000	0.000000	72.050000	1.000000
25th Percentile (Q1)	1.921075e+05	660.815000	1.000000	2.000000	300.995000	17.390000	6.000000	20.755000	7.827500	90.620000	4.000000
50th Percentile (Q2)	2.793230e+05	996.320000	2.000000	3.000000	495.965000	30.385000	7.000000	42.620000	15.625000	94.010000	5.000000
75th Percentile (Q3)	3.918781e+05	1342.292500	3.000000	4.000000	704.012500	43.670000	8.000000	67.232500	25.222500	97.410000	6.000000
Maximum	1.046676e+06	2874.700000	6.000000	7.000000	1631.360000	99.730000	10.000000	178.680000	65.200000	99.900000	10.000000
Median	2.793230e+05	996.320000	2.000000	3.000000	495.965000	30.385000	7.000000	42.620000	15.625000	94.010000	5.000000
Variance	2.255206e+10	181484.930511	1.013826	1.044361	78358.872920	324.912428	3.647835	1009.949427	144.600276	20.298260	3.899702
Range	9.616756e+05	2324.700000	5.000000	6.000000	1630.970000	99.700000	9.000000	178.670000	65.200000	27.850000	9.000000

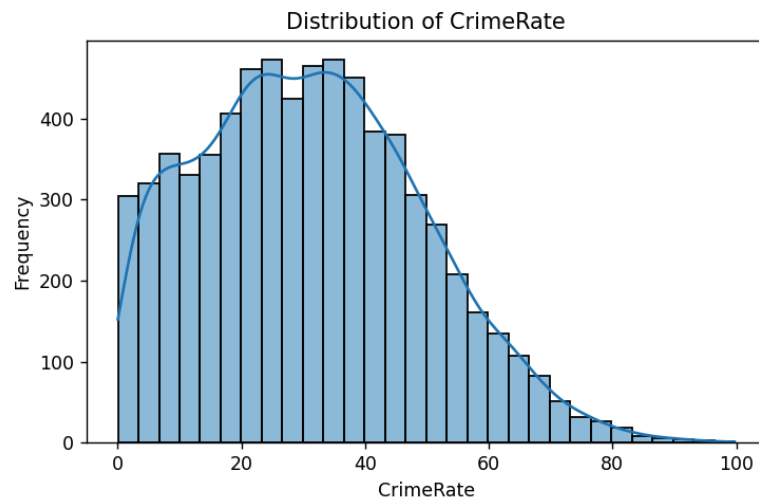
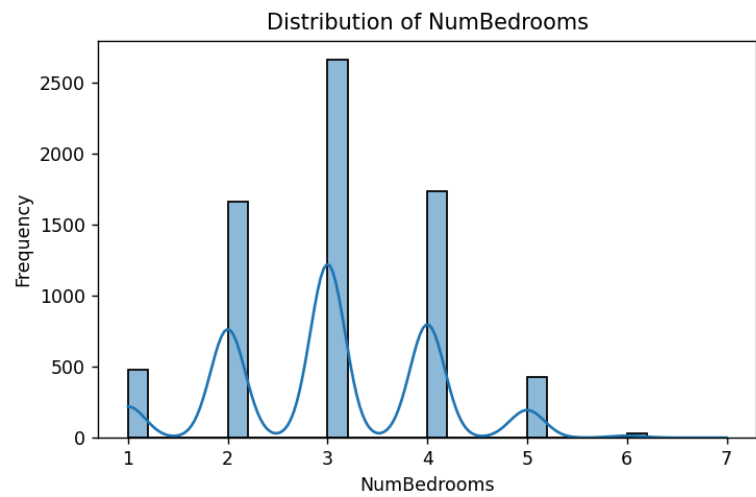
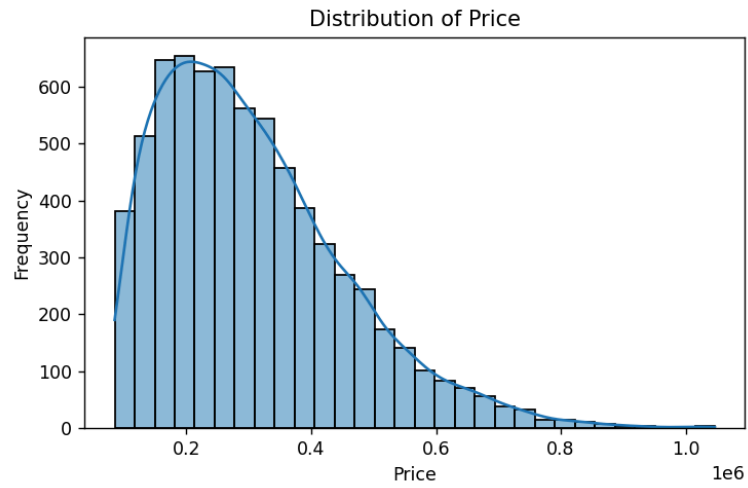
3. Generate univariate and bivariate visualizations of the distributions of the dependent and independent variables from part B1, including the dependent variable in the bivariate visualizations.

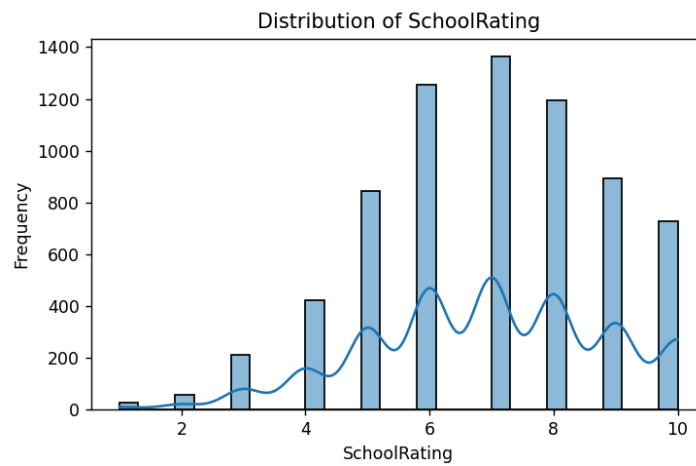
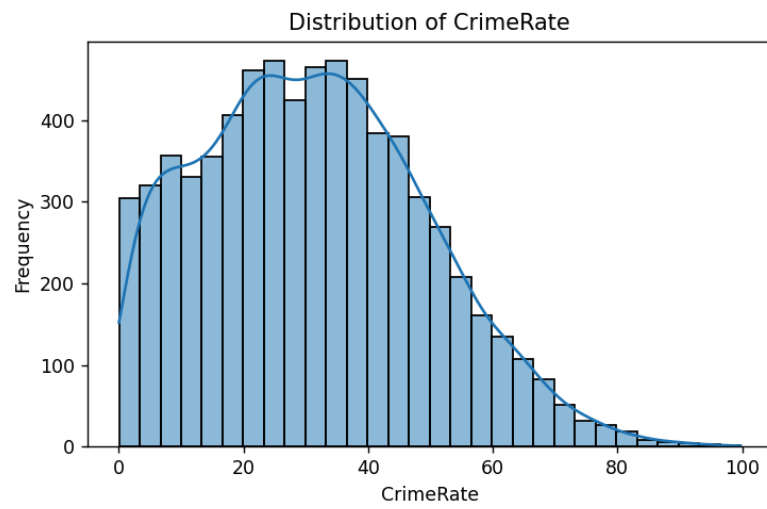
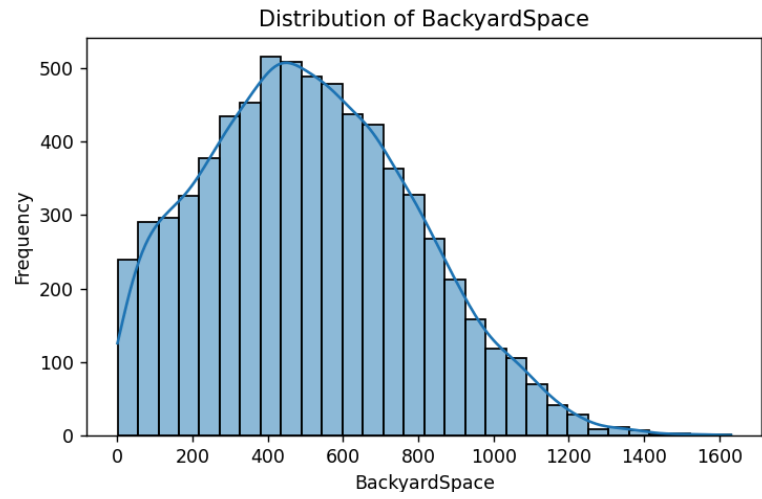
Univariate:

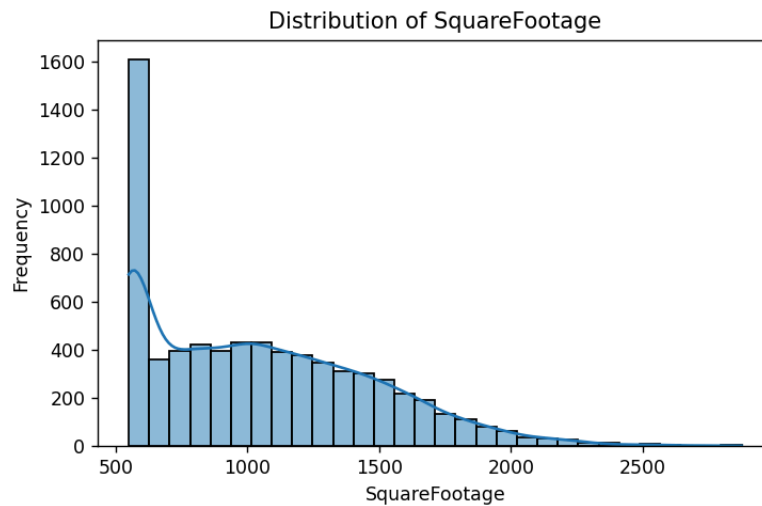
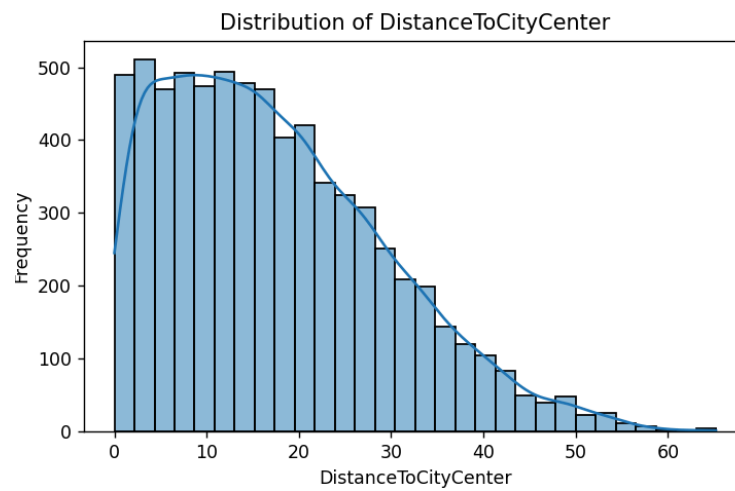
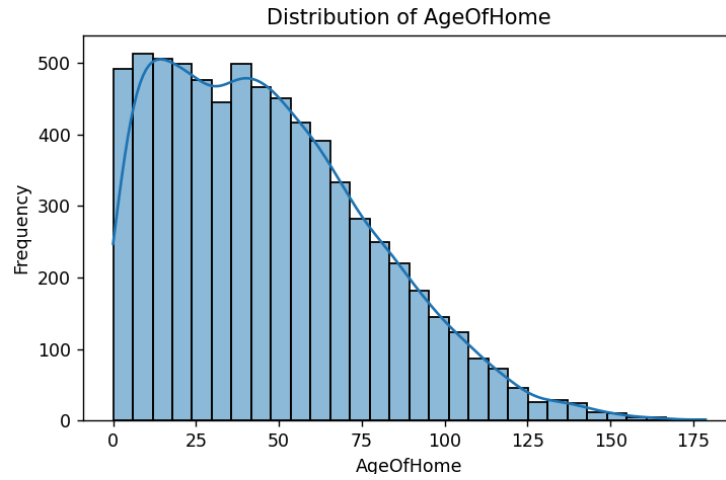
```

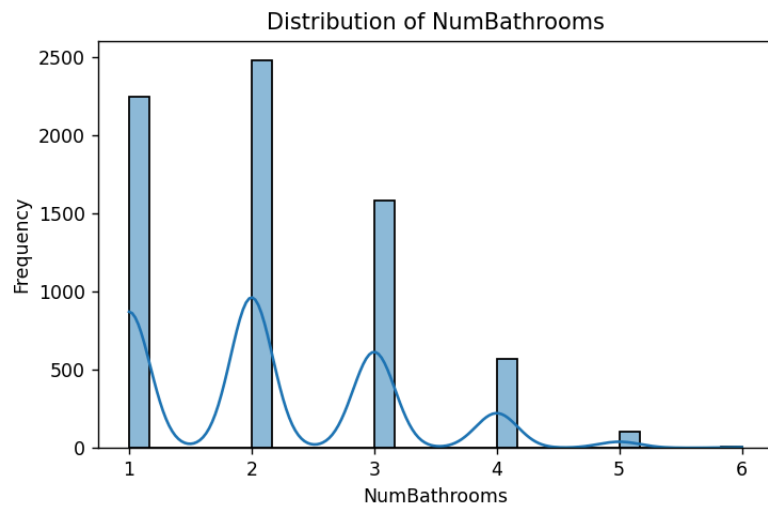
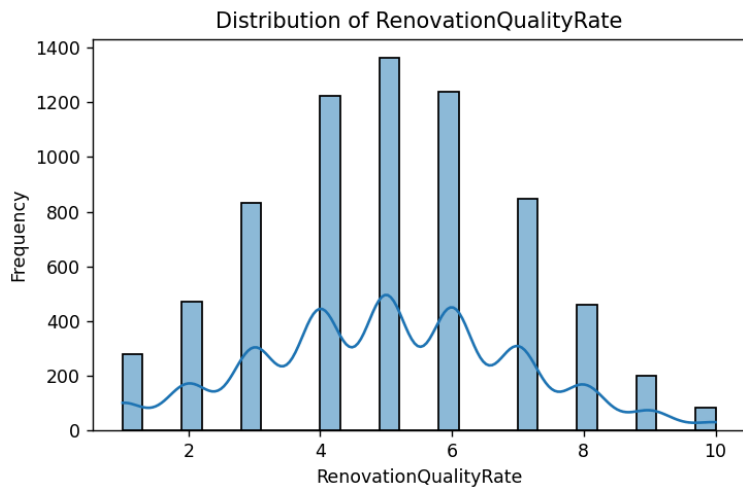
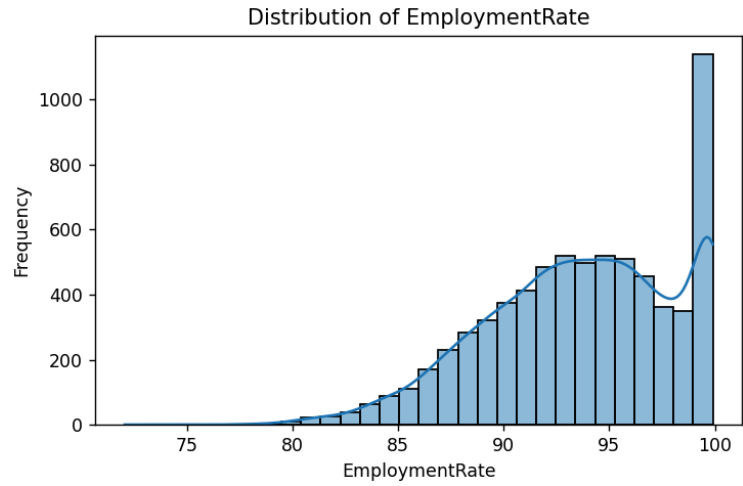
#Plot histograms for each variable
for var in description_vars:
    plt.figure(figsize=(6, 4))
    sns.histplot(data=df, x=var, kde=True, bins=30)
    plt.title(f'Distribution of {var}')
    plt.xlabel(var)
    plt.ylabel("Frequency")
    plt.tight_layout()
    plt.show()

```

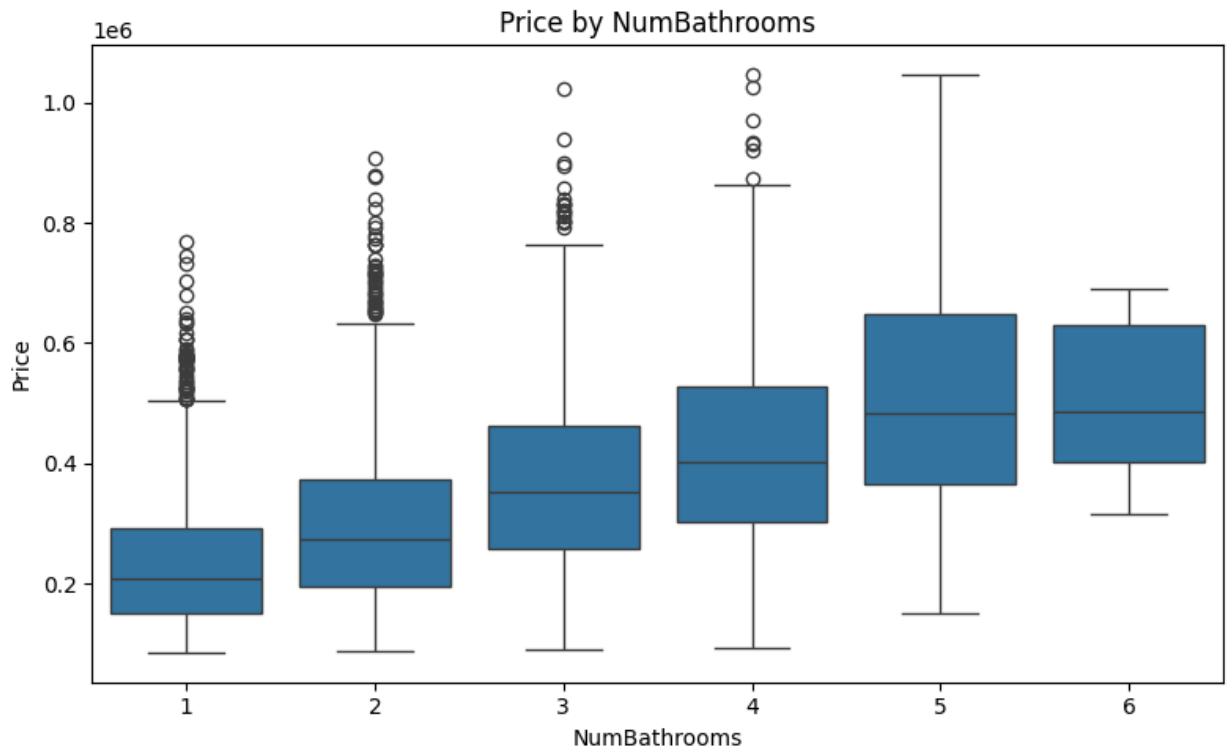
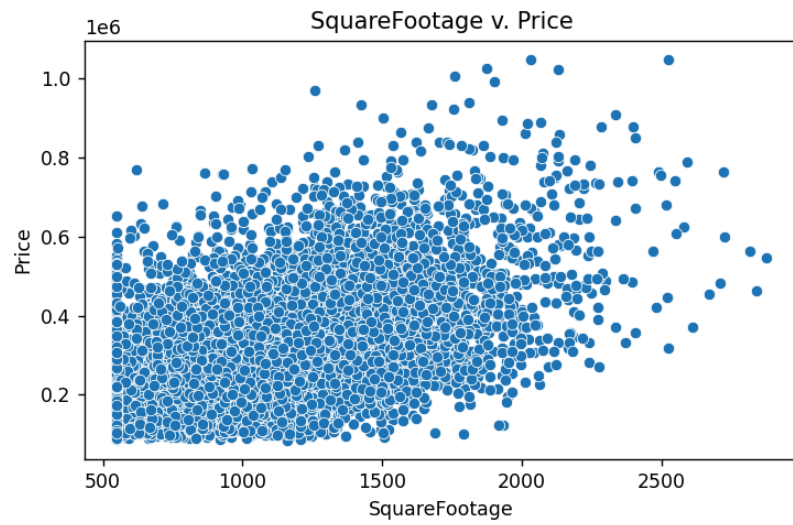


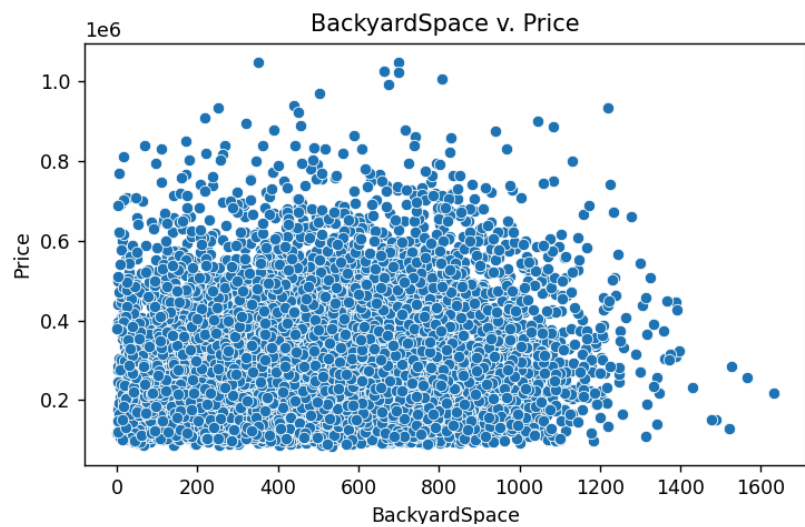
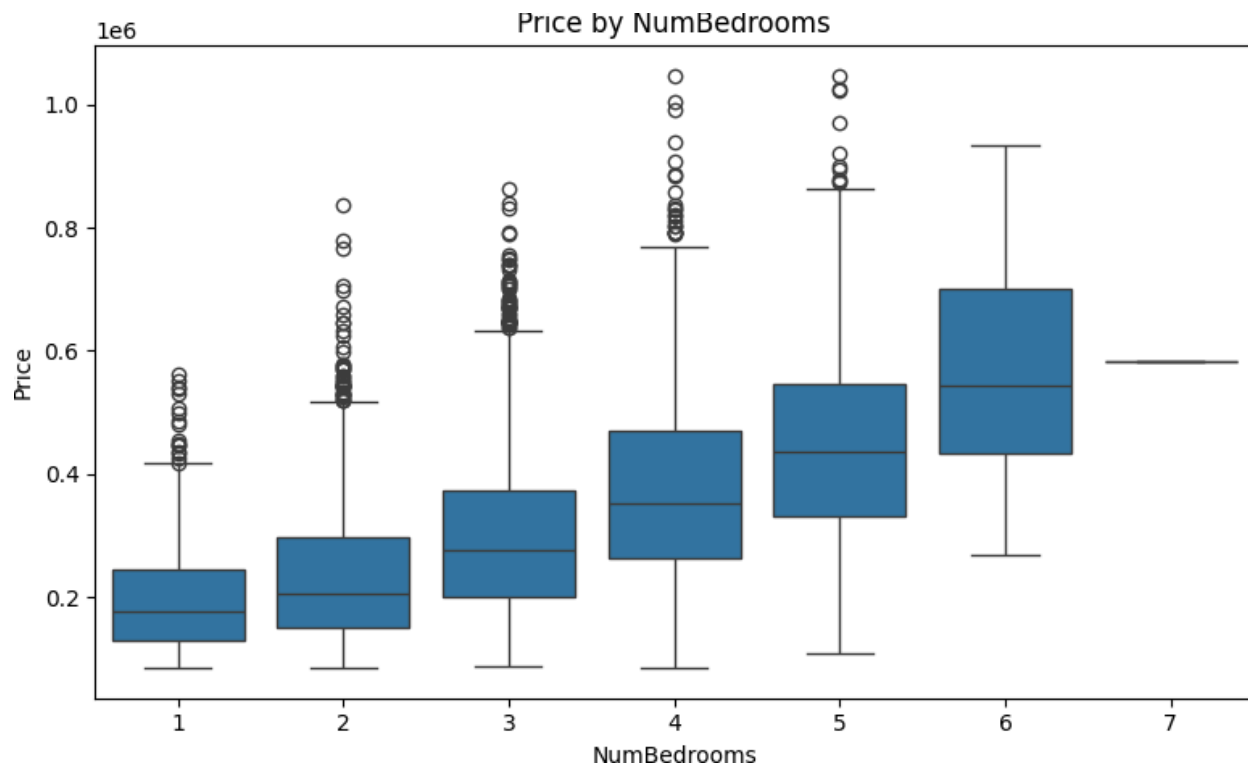


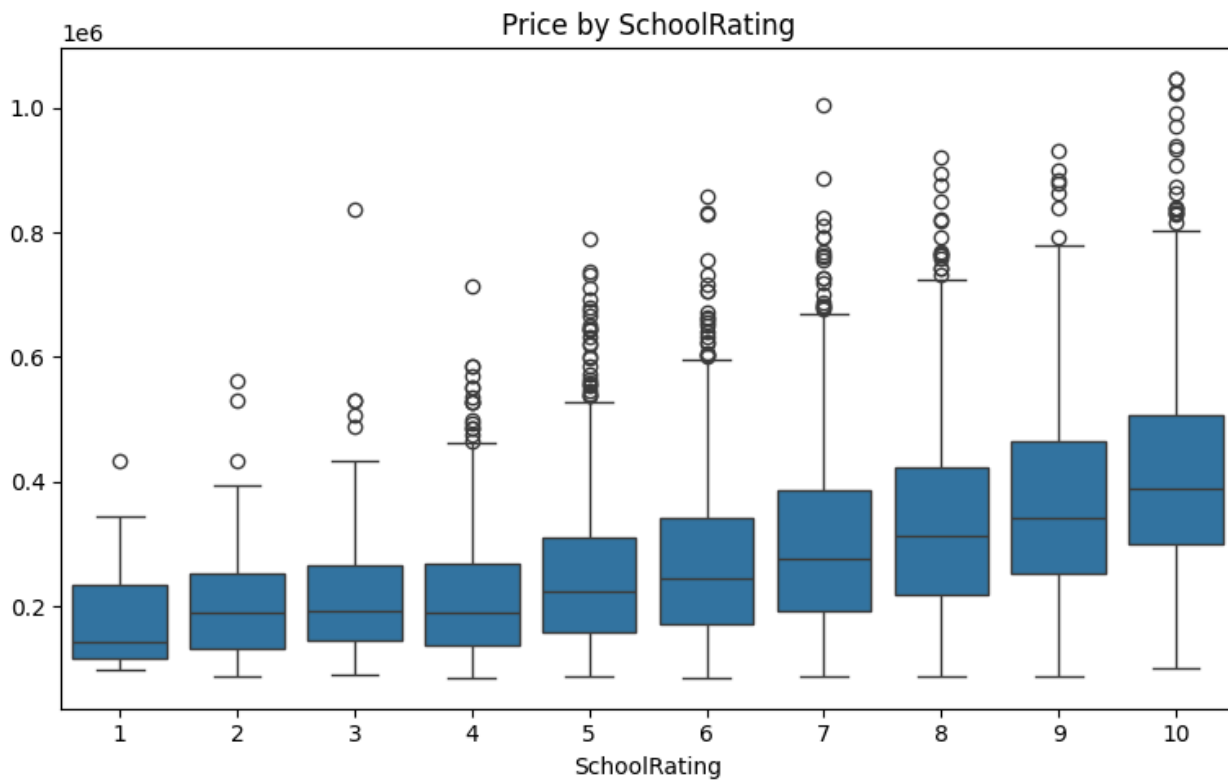
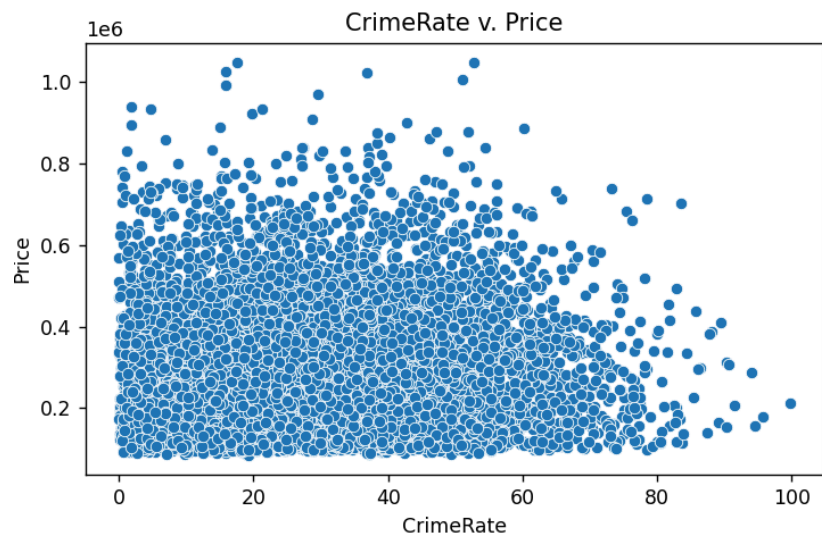


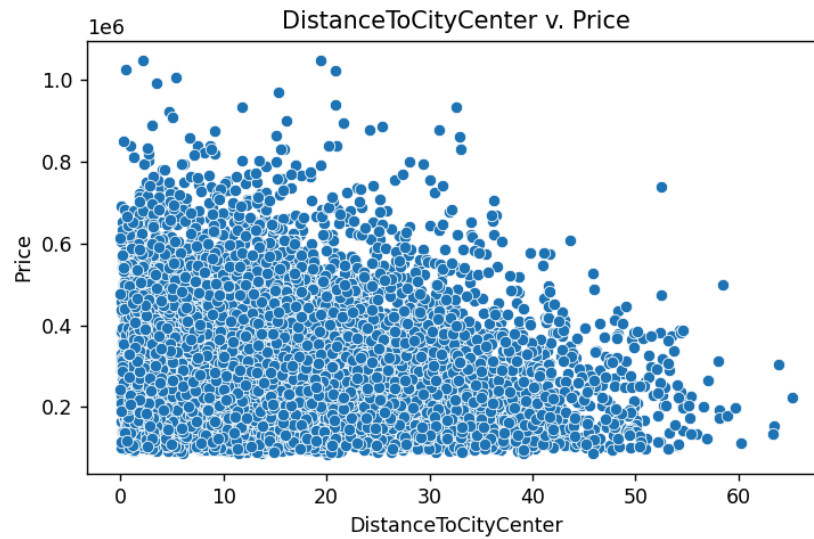
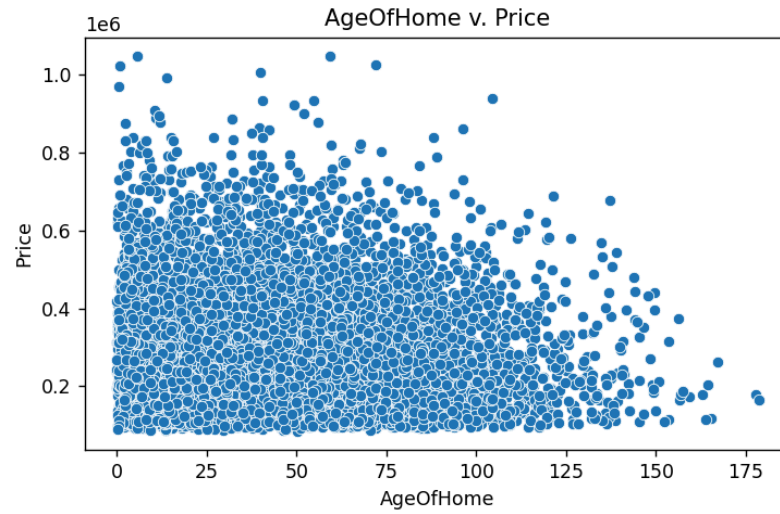


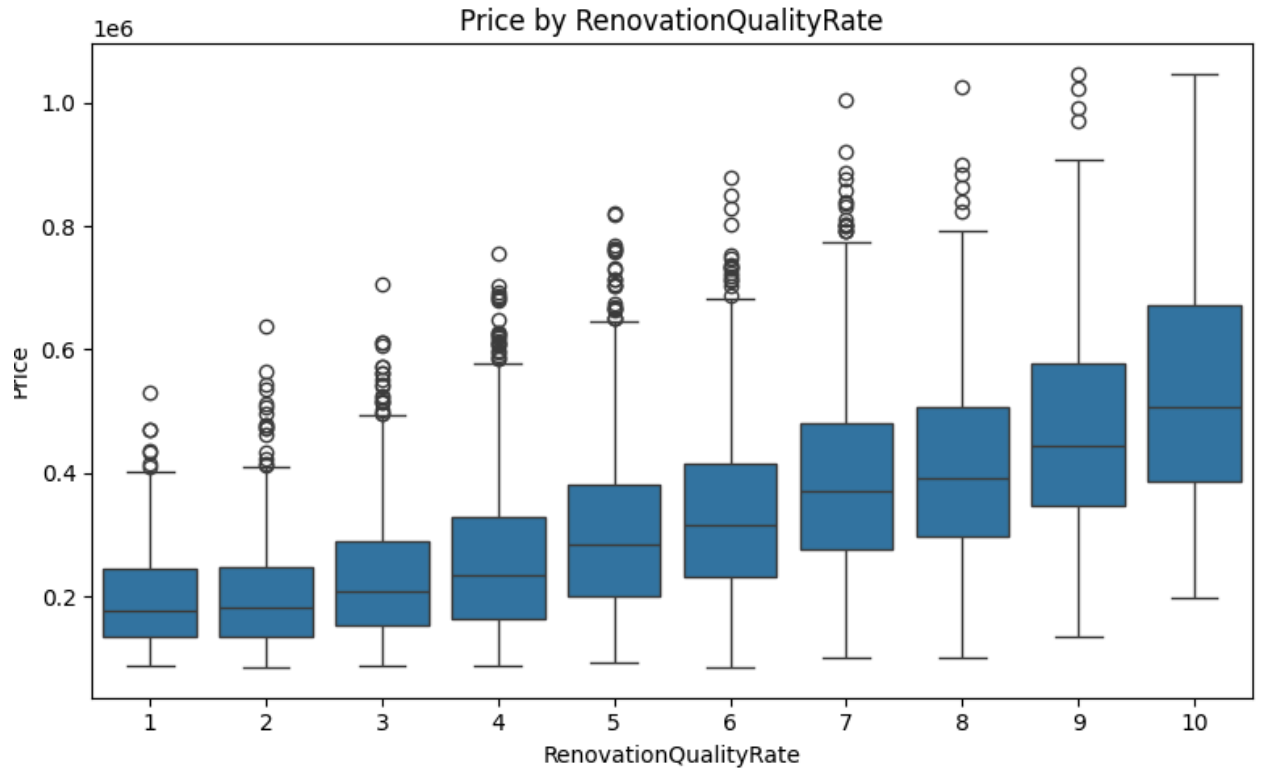
Bivariate:











4. Perform the following data cleaning techniques to prepare the data for modeling and provide a screenshot of the completed steps:

- Handling missing values
- Mitigating outliers
- Encoding categorical variables, if used

Missing Values:

```

110 #Count number of empty cells in each column of dataset before cleaning
111 missing_values_per_column = relevant_df.isna().sum()
112 print(missing_values_per_column)
113
114 #Drop rows with missing values
115 relevant_df=relevant_df.dropna()
116
117 #Count number of empty cells in each column of dataset after cleaning
118 missing_values_per_column = relevant_df.isna().sum()
119 print(missing_values_per_column)
120

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

SquareFootage	0
NumBathrooms	0
NumBedrooms	0
BackyardSpace	0
CrimeRate	0
SchoolRating	0
AgeOfHome	0
DistanceToCityCenter	0
EmploymentRate	0
RenovationQualityRate	0
dtype: int64	
Price	0
SquareFootage	0
NumBathrooms	0
NumBedrooms	0
BackyardSpace	0
CrimeRate	0
SchoolRating	0
AgeOfHome	0
DistanceToCityCenter	0
EmploymentRate	0
RenovationQualityRate	0
dtype: int64	

Outliers:

```

121 #Identify outliers
122 col_mean = relevant_df.mean()
123 col_std = relevant_df.std()
124
125 zscore = (relevant_df-col_mean)/col_std
126
127 #Count outliers before removal
128 outliers_before = (zscore.abs() > 3).any(axis=1).sum()
129 print(f"Number of outlier rows before removal: {outliers_before}")
130 print(f"Total rows before removal: {len(relevant_df)}")
131
132 #Create dataframe without outliers
133 clean_df = relevant_df[(zscore.abs()< 3).all(axis=1)]
134
135 #Count outliers after removal
136 outliers_after = (zscore.abs() > 3).any(axis=1).sum(axis=0)
137 print(f"\nNumber of outlier rows after removal: {0}")
138 print(f"Total rows after removal: {len(clean_df)}")
139

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Number of outlier rows before removal: 243
Total rows before removal: 7000

Number of outlier rows after removal: 0
Total rows after removal: 6757

```

- C. Perform the data analysis and report on the results by doing the following using Python or R:
1. Split the data into **two** datasets, with a larger percentage assigned to the training dataset and a smaller percentage assigned to the test dataset. Provide at least **one** screenshot of the code used to split the data into training and testing datasets.

```

90 from sklearn.model_selection import train_test_split
91 train_df, test_df = train_test_split(clean_df, test_size=0.3, random_state=83)

```

2. Use the training dataset to create a model using multiple linear regression as a statistical method.

```

#Split data into training set and test set
from sklearn.model_selection import train_test_split
train_df, test_df = train_test_split(clean_df, test_size=0.3, random_state=83)

#Defining independent inputs and dependent output in training set
x_train = train_df.drop(columns=['Price'])
y_train = train_df['Price']

#Defining independent inputs and dependent output in training set
x_test = test_df.drop(columns=['Price'])
y_test = test_df['Price']

#Perform Multiple Linear Regression and print summary of fit
x_train_sm = sm.add_constant(x_train)
model_sm = sm.OLS(y_train, x_train_sm).fit()
print(model_sm.summary())

```

3. Create a screenshot of the summary of the initial linear regression model or the following extracted model parameters:

- adjusted R^2
 - R^2
 - F statistics
- probability F statistics
- coefficient estimates
- p -value of each independent variable

OLS Regression Results						
=====						
Dep. Variable:	Price	R-squared:	0.572			
Model:	OLS	Adj. R-squared:	0.571			
Method:	Least Squares	F-statistic:	630.1			
Date:	Tue, 10 Mar 2026	Prob (F-statistic):	0.00			
Time:	10:08:10	Log-Likelihood:	-60765.			
No. Observations:	4729	AIC:	1.216e+05			
Df Residuals:	4718	BIC:	1.216e+05			
Df Model:	10					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	-1.047e+05	2.97e+04	-3.528	0.000	-1.63e+05	-4.65e+04
SquareFootage	135.7953	3.545	38.301	0.000	128.844	142.746
NumBathrooms	4.465e+04	1445.231	30.896	0.000	4.18e+04	4.75e+04
NumBedrooms	5.059e+04	1452.054	34.842	0.000	4.77e+04	5.34e+04
BackyardSpace	-7.8242	4.926	-1.588	0.112	-17.482	1.834
CrimeRate	-82.0136	76.971	-1.066	0.287	-232.912	68.885
SchoolRating	-81.4256	874.545	-0.093	0.926	-1795.943	1633.091
AgeOfHome	-134.0910	44.157	-3.037	0.002	-220.659	-47.523
DistanceToCityCenter	-454.9983	118.344	-3.845	0.000	-687.009	-222.988
EmploymentRate	-57.0024	312.387	-0.182	0.855	-669.427	555.422
RenovationQualityRate	9301.6546	841.013	11.060	0.000	7652.877	1.1e+04
=====						
Omnibus:	247.968	Durbin-Watson:	1.999			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	287.736			
Skew:	0.583	Prob(JB):	3.30e-63			
=====						

4. Optimize the regression model using a process of your selection (e.g., backward stepwise elimination, recursive selection, etc.).

```

#Build Backward Stepwise Function
def backward_stepwise(x, y):
    cols = list(x.columns)

    while True:
        #Fit the model
        x_sm = sm.add_constant(x[cols])
        model = sm.OLS(y, x_sm).fit()

        #Get p-values, excluding the constant
        pvalues = model.pvalues.drop('const')
        max_pvalue = pvalues.max()

        #If all p-values are below 0.05, stop
        if max_pvalue < 0.05:
            print("Final model reached.")
            break

        #Otherwise, remove the variable with the highest p-value
        removed = pvalues.idxmax()
        print(f"Removing: {removed} (p-value: {max_pvalue:.4f})")
        cols.remove(removed)

    print(model.summary())
    return model

# Perform backward stepwise elimination on training data
final_model = backward_stepwise(x_train, y_train)

```

5. Create a screenshot of the summary of the optimized model or the following extracted model parameters:

- adjusted R2
- R2
- F statistics
- probability F statistics
- coefficient estimates
- p-value of each independent variable

OLS Regression Results						
=====						
Dep. Variable:	Price	R-squared:	0.572			
Model:	OLS	Adj. R-squared:	0.571			
Method:	Least Squares	F-statistic:	1050.			
Date:	Tue, 10 Mar 2026	Prob (F-statistic):	0.00			
Time:	12:07:36	Log-Likelihood:	-60767.			
No. Observations:	4729	AIC:	1.215e+05			
Df Residuals:	4722	BIC:	1.216e+05			
Df Model:	6					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	-1.161e+05	6992.117	-16.601	0.000	-1.3e+05	-1.02e+05
SquareFootage	135.4596	3.512	38.571	0.000	128.575	142.345
NumBathrooms	4.455e+04	1437.075	31.000	0.000	4.17e+04	4.74e+04
NumBedrooms	5.053e+04	1391.320	36.321	0.000	4.78e+04	5.33e+04
AgeOfHome	-133.5542	44.069	-3.031	0.002	-219.950	-47.159
DistanceToCityCenter	-457.0908	117.875	-3.878	0.000	-688.181	-226.001
RenovationQualityRate	9239.8850	789.639	11.701	0.000	7691.824	1.08e+04
=====						
Omnibus:	246.908	Durbin-Watson:	2.000			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	286.288			
Skew:	0.582	Prob(JB):	6.81e-63			
Kurtosis:	3.317	Cond. No.	5.90e+03			
=====						

6. Run the prediction on the test dataset using the variables from the optimized regression model to give the prediction model accuracy based on the root mean squared error (RMSE).
Provide a screenshot of the results.

```

#Filter test data to only the 6 significant variables
x_test_optimized = x_test[
    ['SquareFootage',
     'NumBathrooms',
     'NumBedrooms',
     'AgeOfHome',
     'DistanceToCityCenter',
     'RenovationQualityRate']]

#Add constant to match the trained model
x_test_sm = sm.add_constant(x_test_optimized)

#Generate predictions
y_pred = final_model.predict(x_test_sm)

#Calculate RMSE
from sklearn.metrics import mean_squared_error

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print(f"RMSE: {rmse:.2f}")

```

RMSE: 92392.50

D. Summarize your data analysis by doing the following:

1. *List the packages or libraries you have chosen for Python or R and justify how each item on the list supports the analysis.*

In Python I used the following libraries and packages:

Pandas - Pandas is required to import the csv and organize it into a dataframe

Matplotlib - Matplotlib builds all of the structures for the graphs needed in this analysis

Seaborn - Seaborn takes the structures that Matplotlib and creates a visually appealing graphic

Statsmodel.api - Statsmodels.api provides the engine for our entire regression process. It is required to perform the multiple linear regression and the backward stepwise elimination

SciKitLearn - Model_Selection package was necessary to split our data randomly into a test set and a training set.

Metrics package was used to find the root mean squared error in our data.

2. Discuss the method used to optimize the model, providing justification for the approach.

I used backward stepwise elimination to optimize the model. This was the best method for several reasons. First, we started with only 10 independent variables, so the iterative nature of the backward stepwise elimination isn't very resource/time intensive. Secondly, the initial regression showed four variables with high p-values, and backward stepwise elimination is a clear way to remove those variables. Thirdly, the backward stepwise elimination also accounts for shared power between variables and preserves those interactions. Finally, after the backward stepwise elimination, the coefficients of the other variables barely changed and the R-squared value stayed exactly the same, implying that the variables that were removed were insignificant and that the optimization was effective.

3. Complete the following for the optimized model:

a. Create a regression equation by using the coefficients from the optimized model.

$$\text{Price} = -116100 + (135.4596 * \text{SquareFootage}) + (44550 * \text{NumBathrooms}) + (50530 * \text{NumBedrooms}) + (-133.5542 * \text{AgeOfHome}) + (-457.0908 * \text{DistanceToCityCenter}) + (9239.8850 * \text{RenovationQualityRate})$$

b. Provide an interpretation for each coefficient in the regression equation.

Based on the equation, we start at a base price of -\$116,100. Then we add \$135.46 for every square foot within the house. Then, we add \$44,500 for each bathroom the house has. Then, we add \$50,530 for each bedroom the house has. Next, we subtract \$133.55 from the price per year that the house has been built. Then, we subtract \$457.09 for every unit of distance away from the city center. Lastly, we add \$9239.89 for every point on the house's renovation quality rating.

c. Discuss the adjusted R².

The adjusted R-Squared value of 0.571, which accounts for the number of predictors in the model, tells us that about 57.1% of the variations in housing prices are explained by our model. That means that 42.9% of the variations are not described by our model and are probably caused by variables not included in our analysis. This means our model has moderate predicting power, but it is not highly precise.

d. Discuss the accuracy of the optimized model with that of the test dataset based on the RMSE.

Our root-mean-squared error was 92392.50. This tells us that, on average, the predicted housing price from our model is \$92392.50 different from the actual price of the house. The mean price of a house within our test set was \$300123.43. This means that, on average, our model's predicted price is about 30.8% off of the actual price. This aligns with the conclusions from the R-squared value that our optimized model is moderately accurate, but not highly precise.

4. Summarize at least **two** assumptions of multiple linear regression.

Two assumptions of multiple linear regression:

There are no outliers present in the data

There are all observations are independent of one another, meaning one house's price can't be affecting the price of another house

5. Provide evidence that the assumptions from part D4 were verified by providing code snippets or screenshots.

Outliers are specifically handled in the data in the following code

```
80 #Identify outliers
81 col_mean = relevant_df.mean()
82 col_std = relevant_df.std()
83
84 zscore = (relevant_df-col_mean)/col_std
85
86 #Create dataframe without outliers
87 clean_df = relevant_df[(zscore.abs()< 3).all(axis=1)]
```

The independence of our observations is confirmed by the Durbin-Watson value of 2.000 in our results

No. Observations:	4729	AIC:	1.215e+05			
Df Residuals:	4722	BIC:	1.216e+05			
Df Model:	6					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	-1.161e+05	6992.117	-16.601	0.000	-1.3e+05	-1.02e+05
SquareFootage	135.4596	3.512	38.571	0.000	128.575	142.345
NumBathrooms	4.455e+04	1437.075	31.000	0.000	4.17e+04	4.74e+04
NumBedrooms	5.053e+04	1391.320	36.321	0.000	4.78e+04	5.33e+04
AgeOfHome	-133.5542	44.069	-3.031	0.002	-219.950	-47.159
4						
=====						
Omnibus:	246.908	Durbin-Watson:	2.000			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	286.288			
Skew:	0.582	Prob(JB):	6.81e-63			
Kurtosis:	3.317	Cond. No.	5.90e+03			
=====						

6. Discuss a limitation of your analysis.

The biggest limitation of my analysis is the accuracy. With an R-Squared value of 0.571, 42.9% of housing prices are not accurately explained by our model. This means that there are other factors, not included in our model, that have a significant impact on housing prices. We would need to identify other significant variables and introduce them into our model in order to increase the predictive power of our analysis.

7. Recommend a course of action for the real-world organizational situation from part A1 based on the results of your data analysis.

Based on my analysis, I recommend that future models prioritize square footage, renovation quality, number of bedrooms and bathrooms, and distance from city center when predicting housing price. These variables were statistically significant and provided the greatest value in forecasting outcomes.

In terms of my original question, it is evident that investing in renovation can have a very significant effect on housing prices. It is also evident from the data that accurately assessing factors like square footage and the number of bedrooms and bathrooms are vital in accurately leading buyers towards fair pricing.

F. Acknowledge reference sources used to support the Python or R code application. All references listed should also include an in-text citation in the code annotation. Be sure the sources are reliable. If no sources were used for coding, state, "No sources used."

No Sources Used

G. Acknowledge sources, using in-text citations and references, for content that is quoted, paraphrased, or summarized.

No Sources Used